

Examen Info 4
Algorithmique et structure de données
Durée 1h 30

Exercice 1 : ((1. 1pt), (2. 2pts), (3. 2pts), (4. 2pts), (5. 2pts), (6. 2pts))

Considérons une pile statique d'entiers naturels représentée par tableau de taille maximale 20.

1. Définir la structure de données (enregistrement *pile_e*) qui représente cette pile.
2. Ecrire une fonction *pile_e * pile_init()* qui initialise cette pile.
3. Ecrire une fonction *int pile_vide(pile_e *pp)* qui teste si cette pile vide ou pas.
4. Ecrire une fonction *int pile_pleine(pile_e *pp)* qui teste si cette pile est pleine ou pas.
5. Ecrire une fonction *void empiler(int e, pile_e *pp)* qui permet d'ajouter un entier dans cette pile.
6. Ecrire une fonction *int depiler(pile_e *pp)* qui permet d'extraire un entier de cette pile.

Exercice 3 : ((1. 3 pts), (2. 3 pts), (3. 3 pts))

Considérant la représentation d'un texte par une liste chaînée dynamique dont la structure de données est la suivante :

```
typedef struct noeud { char mot[20];
```

```
    noeud *suivant ; };
```

```
noeud * liste,
```

1. Ecrire une fonction itérative qui retourne le nombre des mots constituant le texte.
2. Ecrire une fonction récursive qui retourne le nombre des mots constituant le texte.
3. Ecrire une fonction qui reçoit un mot en argument et qui retourne 0 si ce mot n'existe pas dans la texte et 1 si ce mot existe dans le texte.

\$-Fin-\$

Examen : Algorithmique et Structures de Données

Durée : 1 h 30

ordinaire

Questions : (2 pts)

- ✓ 1) Citer deux représentations différentes d'une file statique. Donner un exemple de structures de données pour définir chacune de ces représentations.
- ✓ 2) Quelle est la discipline suivie par une pile et celle suivie par une file?

Exercice 1 : (4 pts)

- 1) Écrire une fonction qui permet d'allouer d'une façon dynamique de l'espace mémoire pour une matrice carrée de réels.
- 2) Écrire une fonction qui permet de libérer de l'espace mémoire dynamiquement alloué pour une matrice carrée de réels.

Exercice 2 : (14 pts)

On considère une structure de liste chaînée dynamique définie de la façon suivante :

```
typedef struct nœud { int n;  
                    nœud *suivant};  
  
nœud *liste;      /* Variable globale : pointeur externe sur la liste */
```

- 1) Écrire une fonction itérative qui retourne le nombre des entiers constituant le ^{liste} ~~texte~~.
- 2) Écrire une fonction récursive qui retourne le nombre des entiers constituant le ^{liste} ~~texte~~.
- 3) Écrire une fonction qui reçoit un entier en argument et qui retourne 0 si cet entier n'existe pas dans la liste et 1 s'il existe.
- 4) Écrire une fonction qui reçoit un entier en argument et qui l'ajoute à la fin de la liste.
- 5) Écrire une fonction qui reçoit un entier en argument et qui supprime le premier nœud contenant cet entier.

-Fin-

Question: (Page 50)

1) une file est représenté par un enregistrement contenant les champs suivants:

Tableau

- indice du premier élément

- indice du dernier élément.

- File d'attente devant un bus, devant un guichet automatique, etc...

2) - Une file suit la discipline FIFO (First In, first out).

- une pile suit une discipline LIFO (Last in, first out).

Ex 1:

1) float * Mat_alloc_float(int n)

{ float * mat

int i;

mat = (float **) malloc(n * sizeof(float))

if (mat == NULL) {

printf("Mémoire insuffisante\n");

exit;

for (i = 0; i < n; i++) {

mat[i] = (float *) malloc(n * sizeof(float));

if (mat[i] == NULL) {

printf(stderr, "Mémoire insuffisante\n");

exit;

}

return mat;

}

2)

```
void mat free(float *mat, int n)
{
    int i;
    for (i = 0; i < n; i++)
    { free(mat[i]); }
    free(mat);
    return;
}
```

Allocation dynamique pour un tableau à 1 dimension

```
int n, *tab;
printf("taille du tableau: "); scanf("%d", &n);
tab = (*int*) malloc (n * sizeof(int));
malloc (n * sizeof(int))
```

Ex 2:

```
1) int nombreEntier ( )  
{  
    noeud * a;  
    int nbr = 0;  
    a = Piste;  
    while (a != NULL)  
    {  
        ++nbr;  
        a = a -> suivants;  
    }  
    return (nbr);  
}
```

```
2) int nombreEntierRec ( noeud * PP )  
{  
    if ( PP == NULL )  
        return 0;  
    else  
        return ( 1 + nombreEntierRec ( PP -> suivants ) );  
}
```

3) int recher-Piste (int x)

```
{ noeud * a;
```

```
  a = Piste;
```

```
  if (a == NULL) return 0;
```

```
  else
```

```
    { if (a->n == x) return 1;
```

```
      else
```

```
        { return (recher-Piste(x));
```

```
      }
```

```
    }
```

4) void Ajout_Fin (int x)

```
{
```

```
  noeud * Save, *b;
```

```
  b = (noeud *) malloc (sizeof (noeud));
```

```
  b->n = x;
```

```
  b->suivant = NULL;
```

```
  if (Piste == NULL) { Piste = b;
```

```
  else {
```

```
    Save = Piste;
```

```
    while (Save->suivant != NULL)
```

```
    {
```

```
      Save = Save->suivant;
```

```
      Save->suivant = b;
```

```
    }
```

```
  }
```

```
  return 0;
```



```

5) void supprimer (int m, Noeud *Piste)
    Noeud *P = Piste, PP = NULL, q;
    while (P != NULL && (*P != m) == m)
    {
        PP = P;
        P = P -> suivant;
    }
    if (PP == NULL)
    {
        q = P;
        P = P -> suivant;
        Piste = P;
        free(q);
    }
    else
    {
        if (P == NULL)
        {
            PP -> suivant = P -> suivant;
        }
        else
        {
            PP -> suivant = q;
        }
    }
    return (0);
}

```



```

5) void supprimer (int m, Noeud* Piste)
    Noeud* P = Piste; PP = NULL; q;
    while (P != NULL && (*P != m) = m)
    {
        PP = P;
        P = P -> suivant;
    }
    if (PP == NULL)
    {
        q = P;
        P = P -> suivant;
        Piste = P;
        free(q);
    }
    else
    {
        if (P != NULL)
        {
            PP -> suivant = P -> suivant;
        }
        else
        {
            PP -> suivant = q;
        }
    }
    return (0);
}

```

Examen de Rattrapage : Algorithmique et Structures de Données

Durée : 1 h 30

4 Questions : (2 pts)

- ✓ 1) Citer deux représentations différentes d'une pile statique. Donner un exemple de structures de données pour définir la représentation d'une pile statique.
- ✓ 2) Quelle est la discipline suivie par une file et celle suivie par une pile?

Exercice 1 : (4 pts)

- 1) Écrire une fonction qui permet d'allouer d'une façon dynamique de l'espace mémoire pour une matrice réelle dont le nombre de lignes nl et le nombre de colonnes nc sont passés en argument.
- ✓ 2) Écrire une fonction qui permet de libérer de l'espace mémoire dynamiquement alloué pour un tableau d'entiers de dimension 1.

Exercice 2 : (14 pts)

On considère une structure de liste chaînée dynamique définie de la façon suivante :

```
typedef struct nœud { float real ;  
                     nœud *suivant } ;  
nœud *liste;        /* Variable globale : pointeur externe sur la liste */
```

- 1) Ecrire une fonction qui reçoit un réel en argument et retourne l'occurrence de ce réel dans la liste.
- 2) Ecrire une fonction récursive qui retourne la longueur de la liste.
- 3) Ecrire une fonction qui reçoit un réel en argument et qui retourne 1 si ce réel existe dans la liste et 0 s'il n'existe pas dans la liste.
- 4) Ecrire une fonction qui reçoit un réel en argument et qui l'ajoute à la fin de la liste.
- 5) Ecrire une fonction qui reçoit un réel en argument et qui change la valeur du premier nœud de la liste contenant ce réel à 2.01.

-Fin-

Examen
Algorithmique et Structures de Données
Durée 1h 30

Exercice 1 : (2.5 pts)

- ② Ecrire une fonction qui permet d'allouer d'une façon dynamique de l'espace mémoire pour une matrice carrée de réels.

Exercice 2 : (2.5 pts)

- √ Ecrire une fonction qui permet de libérer de l'espace mémoire dynamiquement alloué pour un tableau de trois dimensions dont les éléments sont des réels.

Exercice 3 : (15 pts (3 pts pour chaque question))

On considère la représentation d'un texte par une liste chaînée dont la structure de données est donnée par :

```
typedef struct noeud { char mot[20];  
                        noeud *suivant ; };
```

noeud * liste,

1. Ecrire une fonction itérative qui retourne le nombre des mots constituant le texte.
2. Ecrire une fonction récursive qui retourne le nombre des mots constituant le texte.
3. Ecrire une fonction qui reçoit un mot en argument et qui retourne 0 si ce mot n'existe pas dans le texte et 1 si ce mot existe dans le texte.
4. Ecrire une fonction qui reçoit un mot en argument et qui l'ajoute à la fin de la liste.
5. Ecrire une fonction qui reçoit un mot en argument et qui supprime le premier noeud contenant ce mot.

Fin

Ex 3 :

```
1) int calcul_iterative ( Noeud* liste )  
{  
    Noeud* a; liste  
    int cp=0; // le compteur  
    if (a == NULL)  
        return 0;  
    while (a != NULL) {  
        cp++;  
        a = a->suivant;  
    }  
    return cp;  
}
```

```
2) int calcul_Recursive ( Noeud* liste )  
    Noeud* a;  
    if (a == NULL) return 0;  
    else return (1 + calcul_Recursive(  
        (a->suivant)
```



```

3) int recherche (char c[20], Noeud *liste)
    {
        while (a != NULL) {
            int i = 0;
            while ((^a).mot[i] == c[i] & i < 20)
                i++;
            if (i >= 20) return 1;
            a = a -> suivant;
        }
        if (a == NULL) return 0;
        return 1;
    }
}

```

```

4) Noeud *ajouter_fin(char c[20], Noeud *liste)
{
    Noeud *a = liste;
    Noeud *q = (Noeud *) malloc(sizeof(Noeud));
    int i;
    for(i=0; i<20; i++)
    {
        (*q).mot[i] = c[i];
    }
    q->suivant = NULL;
    if(p == NULL) return q;
    while(a->suivant != NULL)
    {
        a = a->suivant;
    }
    a = a->suivant;
    a->suivant = q;
    return liste;
}

```

5) void supprimer(char c[20], Noeud *liste)

```
{  
    Noeud *p = liste, pp = NULL, q;  
    int i = 0;  
    while ((*p).mot[i] == c[i] && i < 20)  
        while (p != NULL && (*p).mot[i] == c[i])  
        i++;  
    while (p != NULL && (*p).mot[i] == c[i])  
    {  
        pp = p;  
        p = p->suivante;  
    }  
    if (pp == NULL)  
    {  
        q = p;  
        p = p->suivante;  
        liste = p;  
        free(q);  
    }  
    else  
    {  
        if (p != NULL)  
        {  
            pp->suivant = p->suivant;  
        }  
        else  
        {  
            pp->suivant = q;  
        }  
    }  
}
```